ADR-CAMP-01 — Pontuação de equipes como extrato contábil (ledger)

Autor(a): a2648938 — Rafael Moraes Trevizan
Projeto individual: Sistema de Gestão de Campeonatos de Jiu-Jitsu

Status

Proposed

Contexto

Em um campeonato de Jiu-Jitsu, várias áreas de luta (tatames) funcionam ao mesmo tempo. Cada resultado lançado gera pontos para a equipe do vencedor, e cada categoria finalizada gera pontos de pódio (1º, 2º e 3º). O ranking geral de equipes decide o título de "equipe campeã". Por isso ele precisa ser **correto**, **auditável** e **atualizado em tempo real**.

Problema: como armazenar a pontuação das equipes para que:

- lançamentos simultâneos de vários tatames não se sobrescrevam (condição de corrida);
- seja possível corrigir ou anular uma luta, ou reabrir uma categoria, e estornar exatamente os pontos que ela gerou;
- qualquer equipe possa ver **de onde veio** cada ponto (transparência e prevenção de disputas);
- seja impossível uma luta ou colocação pontuar duas vezes.

Por que a decisão é necessária agora: o modelo de pontuação afeta o esquema do banco, os services de resultado, correção e finalização, o ranking e a auditoria. Mudar depois exigiria migrar dados de eventos já realizados.

Restrições:

- banco relacional PostgreSQL já adotado (RNF22);
- operações compostas precisam ser atômicas (RNF09);
- consultas de ranking devem responder em menos de 200 ms (RNF14);
- equipe pequena: a solução não pode exigir infraestrutura extra (filas, brokers, event store).

Requisitos relacionados

Tipo	Requisitos
Funcionais	RF46 (avanço atômico com pontuação), RF50 (estorno automático), RF56 (reabertura com estorno), RF57 (regras configuráveis por evento), **RF58 (extrato, sem saldo estático)**, RF59 (anti-duplicidade), RF60 (ajustes e penalidades), RF61 (ranking em tempo real), RF62 (desempate), RF63 (extrato por equipe)
Não funcionais	RNF08 (validação em duas camadas), **RNF09 (transações ACID)**, **RNF10 (ledger)**, RNF11 (índices anti-duplicação), RNF12 (índices), RNF14 (desempenho)

Tipo	Requisitos
Arquiteturais	Integridade dos dados sob concorrência, rastreabilidade, simplicidade operacional

Alternativas consideradas

A1 — Coluna de saldo na tabela `equipes` (`total_pontos`)

Cada vitória executa `UPDATE equipes SET total_pontos = total_pontos + X`.

- **Vantagens:** ranking trivial (`ORDER BY total_pontos`); leitura muito rápida; modelo fácil de entender.
- **Desvantagens:** perde a origem dos pontos, então não atende RF63; o estorno depende de "lembrar" quanto foi somado; o saldo é por equipe e não por evento, o que exigiria uma tabela `equipe_evento`.
- **Riscos:** dessincronização silenciosa (um bug ou uma correção manual deixa o saldo errado sem jeito de recalcular); disputa de lock na mesma linha quando vários tatames pontuam a mesma equipe.
- **Impactos:** auditoria precisaria reconstruir o histórico só pelos logs.
- **Limitações:** não suporta ajustes ou penalidades com justificativa rastreável.

A2 — Extrato de lançamentos (ledger) com total calculado por `SUM` ☑

Cada fato que gera ponto vira **uma linha** em `pontos_equipes` (evento, equipe, inscrição, luta ou chave de origem, tipo, pontos). O ranking é `SUM(pontos) GROUP BY equipe`.

- **Vantagens:** origem de cada ponto preservada (RF63); total sempre consistente com os lançamentos; estorno preciso (remove ou compensa o lançamento daquela luta ou chave); índices únicos parciais impedem duplicidade **no banco**; `INSERT`s concorrentes não disputam a mesma linha.
- **Desvantagens:** o ranking precisa de agregação a cada consulta; mais linhas no banco.
- **Riscos:** queda de desempenho em eventos muito grandes sem índices adequados.
- **Impactos:** todos os services que pontuam passam a inserir lançamentos dentro da mesma transação do fato de origem.
- **Limitações:** o histórico de lançamentos removidos depende da tabela de auditoria (ver Consequências).

A3 — Tabela de ranking pré-calculada / view materializada

Mantém lançamentos e também uma tabela `ranking` atualizada por trigger ou `REFRESH MATERIALIZED VIEW`.

- **Vantagens:** leitura do ranking O(1); mantém a origem dos pontos.
- **Desvantagens:** duas fontes de verdade; triggers escondem regra de negócio no banco; `REFRESH` não é instantâneo, o que fere o "tempo real" do RF61.
- **Riscos:** inconsistência entre ranking e extrato; deadlocks com triggers sob concorrência.
- **Impactos:** dificulta testes e manutenção (lógica dividida entre Node e SQL).
- **Limitações:** otimização prematura para o volume esperado.

A4 — Event Sourcing completo

Todo o estado (lutas, chaves, pontos) é derivado de um log de eventos imutável, com projeções de leitura.

- **Vantagens:** histórico total e replay; imutabilidade real; ótimo para auditoria forense.
- **Desvantagens:** complexidade muito alta (projeções, versionamento de eventos, consistência eventual); exige reescrever o sistema inteiro.
- **Riscos:** curva de aprendizado e bugs de projeção; consistência eventual é inaceitável para o placar em tempo real.
- **Impactos:** afeta todos os módulos, não só a pontuação.
- **Limitações:** desproporcional ao tamanho da equipe e do problema.

Critérios de decisão

Critério	Peso	A1 Saldo	A2 Ledger	A3 Pré-calculado	A4 Event Sourcing
Integridade sob concorrência	5	2	**5**	3	5
Rastreabilidade da origem dos pontos	5	1	**5**	4	5
Facilidade de estorno/correção	4	2	**5**	3	4
Desempenho do ranking (< 200 ms)	3	5	**4**	5	3
Simplicidade / custo de implementação	4	5	**4**	2	1
Total ponderado		58	**98**	70	79

(Escala 1 a 5; total = soma de nota × peso.)

Decisão

A pontuação das equipes será registrada **exclusivamente** como lançamentos individuais na tabela `pontos_equipes`. **Não existirá** campo de saldo em `equipes`. Concretamente:

1. Tipos permitidos (CHECK no banco): `VITORIA`, `PRIMEIRO_LUGAR`, `SEGUNDO_LUGAR`, `TERCEIRO_LUGAR`, `AJUSTE`, `PENALIDADE` (`010_create_pontos_equipes.sql`).
2. Índices únicos parciais garantem no máximo **1 `VITORIA` por luta** e **1 de cada colocação por chave** (`011_create_indexes.sql`).
3. Os valores de pontos vêm da configuração do evento (`regras_pontuacao`, RF57).
4. Todo lançamento ou estorno ocorre **na mesma transação** do fato que o originou (resultado, correção, finalização ou reabertura).
5. O ranking é sempre calculado por `SUM(pontos)`, filtrado por evento, ordenado por total decrescente e, em seguida, por nome da equipe (RF61, RF62).

Verificação: não existe coluna de pontos em `equipes`; tentar inserir duas `VITORIA` para a mesma `luta_id` retorna erro de unicidade; os scripts `npm run test:pontuacao`, `test:correcao` e `test:ranking` passam; `npm run test:integridade` confere a consistência.

Justificativa

- O requisito mais crítico do domínio é **o ranking não poder estar errado**. Com o ledger, o total é derivado e não armazenado, então não existe um "saldo" que possa dessincronizar.
- Concorrência entre tatames vira uma sequência de `INSERT`s independentes, sem disputa por lock na linha da equipe como aconteceria em A1.
- O estorno exigido por RF50/RF56 fica simples: basta localizar o lançamento pela `luta_id` ou `chave_id`, sem recalcular nada.
- A garantia anti-duplicidade fica **no banco** (índices parciais), não só na aplicação. É a validação em duas camadas do RNF08.
- O volume de dados é pequeno. Um evento grande (cerca de 1.000 atletas) gera alguns milhares de lançamentos, e um `SUM` indexado por `evento_id` responde bem abaixo de 200 ms. Por isso A3 seria otimização prematura e A4, complexidade desnecessária.

Consequências

Positivas

- Ranking sempre consistente e recalculável a qualquer momento (RF61).
- Extrato detalhado por equipe disponível "de graça" (RF63).
- Correções, anulações e reaberturas estornam pontos de forma precisa e atômica (RF50, RF56).
- Ajustes e penalidades manuais convivem com os pontos automáticos no mesmo modelo (RF60).
- Proteção física contra pontuação duplicada, mesmo com bug ou duplo clique (RF59, RNF11).

Negativas

- O ranking custa uma agregação por consulta. Em escala muito maior pode exigir índice composto `(evento_id, equipe_id)` (pedido no RNF12, hoje há índices separados) ou cache.
- **Imutabilidade parcial:** hoje o estorno é feito com `DELETE` do lançamento (`excluirPontoVitoria`, `excluirPontosColocacaoPorChave`) e o histórico fica preservado em `logs_auditoria`. Um ledger "puro" usaria **lançamentos de estorno** (pontos negativos) em vez de remover linhas. Isso é uma evolução possível, mas exigiria rever os índices únicos parciais.
- Todo novo fluxo que pontua precisa lembrar de inserir o lançamento dentro da transação, o que aumenta o acoplamento com o padrão transacional.
- Se uma equipe tiver lançamentos, ela não pode ser excluída (`ON DELETE RESTRICT`).

Impactos sobre outras equipes

> Por ser um **projeto individual**, não há outras equipes. Esta seção lista os **módulos do próprio sistema** afetados pela decisão.

Módulo / área afetada	Impacto
Lançamento de resultados (Módulo 10, `resultadoService`)	Ao finalizar uma luta, insere o lançamento `VITORIA` na mesma transação do avanço de fase (RF46).
Correção e anulação (Módulo 11)	Toda correção ou anulação localiza o lançamento pela `luta_id` e o estorna atomicamente (RF50).
Finalização e pódio (Módulo 12, `finalizacaoService`)	Cria os lançamentos de 1º, 2º e 3º lugar. A edição do pódio e a reabertura removem e recriam esses lançamentos (RF54, RF56).

Módulo / área afetada	Impacto
Ranking e extrato (Módulo 14, `rankingService`)	O ranking passa a ser uma consulta `SUM` e o extrato por equipe é a listagem dos lançamentos (RF61–RF63).
Equipes (Módulo 4)	Equipes com lançamentos não podem ser excluídas (`ON DELETE RESTRICT`).
Auditoria (Módulo 15)	Guarda o histórico dos lançamentos estornados, já que eles são removidos da tabela.
Banco de dados / infraestrutura	Depende de transações ACID e índices únicos parciais, o que exige PostgreSQL.
Testes	Os scripts precisam cobrir duplicidade, estorno após correção e reabertura, e consistência do `SUM`.